

Desenvolvimento para Dispositivos Móveis

Armazenamento de Dados

Profº: Joseph Donald

Contatos:

☎ (83) 98228-8607

📷 @josephdonald

✉ josephdonald.pi@gmail.com

“Se você tem uma maçã e eu tenho outra; e nós trocamos as maçãs, então cada um terá sua maçã. Mas se você tem uma ideia e eu tenho outra, e nós as trocamos; então cada um terá duas ideias.”

George Bernard Shaw



- Um banco de dados é uma ferramenta essencial para armazenar e gerenciar grandes volumes de dados de forma organizada e eficiente.
- O armazenamento de dados em um aplicativo móvel desempenha um papel fundamental na melhoria da experiência do usuário, no desempenho do aplicativo, na personalização, na segurança dos dados e na disponibilidade offline. É uma parte essencial do desenvolvimento de aplicativos móveis bem-sucedidos que atendam às necessidades dos usuários de forma eficiente e confiável.

Armazenamento de dados: Os aplicativos precisam de um local para armazenar e recuperar dados.

Gerenciamento de dados: Um banco de dados permite o gerenciamento centralizado e organizado dos dados. Ele define a estrutura dos dados, regras de validação, restrições e relacionamentos entre diferentes conjuntos de dados. Isso ajuda a manter a integridade e a consistência dos dados, evitando duplicação e inconsistências.

Acesso simultâneo: Com a conexão do aplicativo ao banco de dados, vários usuários ou processos podem acessar e trabalhar nos dados ao mesmo tempo. Isso é crucial em ambientes compartilhados, como sistemas corporativos, onde várias pessoas precisam acessar e atualizar os dados simultaneamente.

Consultas e análises de dados: Os bancos de dados fornecem uma linguagem de consulta poderosa e consolidada, como o SQL (Structured Query Language), que permite aos aplicativos realizar consultas complexas para recuperar dados específicos com eficiência. Isso facilita a extração de informações úteis e a realização de análises em larga escala.

Segurança dos dados: A conexão com um banco de dados permite a implementação de recursos de segurança para proteger os dados sensíveis. Por exemplo, é possível definir permissões de acesso, criptografar dados armazenados e auditar atividades relacionadas aos dados.

Escalabilidade: Um banco de dados adequado permite que os aplicativos sejam dimensionados para atender a um grande número de usuários e um volume crescente de dados. Os bancos de dados modernos são projetados para lidar com carga de trabalho intensa e podem ser configurados para otimizar o desempenho e a escalabilidade.



De que forma posso armazenar dados no App?

Shared Preferences (preferências - arquivos): são geralmente usadas para armazenar e recuperar configurações e informações do usuário de maneira persistente em arquivos. Elas podem ser usadas para salvar preferências de usuário, como configurações, preferências de tema, estado do aplicativo ou qualquer outra informação que precise ser mantida entre as sessões do usuário.

SQLite (banco local): é um banco de dados relacional embutido, rápido e de fácil utilização, amplamente utilizado em aplicativos móveis e outros projetos que exigem armazenamento local de dados. Ele oferece suporte a SQL completo e é uma opção popular para armazenar e gerenciar dados em aplicativos Flutter.

Consumo de APIs (banco externo): A conexão entre os aplicativos e o banco de dados permite que os dados sejam inseridos, recuperados, atualizados e excluídos de maneira segura e confiável.



Utilização da classe **FUTURE**

- As operações para armazenamento de dados são tratadas de forma **assíncrona**, pois:
 - ✓ Acesso ao disco (como no SQLite ou SharedPreferences) pode ser lento.
 - ✓ Escrever ou ler arquivos, mesmo que pequenos, leva tempo.
 - ✓ O sistema precisa manter a interface fluida, sem travar a tela.
- Para tratar os dados assíncronos, utilizamos uma classe chamada **Future**. Esta classe trata um valor que estará disponível no futuro.
- Ele é usado para operações assíncronas, como:
 - ✓ Ler ou gravar no armazenamento (Shared Preferences / SQLite).
 - ✓ Fazer requisições HTTP (consumo de APIs).
 - ✓ Esperar por um tempo ou executar tarefas demoradas.



```
Future<String> buscarDados() async {  
    await Future.delayed(Duration(seconds: 2));  
    return "Dados recebidos!";  
}
```

Future<String>: A função retorna um Future que vai conter uma String no futuro. Ou seja, ela não retorna a String imediatamente, mas promete que um dia retornará esse valor.

buscarDados(): É o nome da função, que simula buscar dados (ex: de um banco de dados ou API).

async: Torna a função assíncrona, permitindo o uso de **await** dentro dela. Indica que a função executa algo que leva tempo, e não bloqueia a execução da UI enquanto espera.

await: Faz o programa esperar esses 2 segundos antes de continuar, sem travar a interface. Essa é uma forma de simular um carregamento, como se estivesse buscando dados de um servidor.

Future.delayed(...): Cria um Future que será concluído após um determinado tempo. Neste caso, 2 segundos de espera.

No Flutter, você pode usar o pacote *shared_preferences* para trabalhar com preferências de forma simples e eficiente. Esse pacote fornece uma API fácil de usar para armazenar e recuperar valores no armazenamento persistente do dispositivo.

Execute o comando: *flutter pub add shared_preferences*

```
s\projeto_preferences> flutter pub add shared_preferences
```

Depois verifique o arquivo
pubspec.yaml

```
! pubspec.yaml
1  name: projeto_preferences
2  description: "A new Flutter project."
3  publish_to: 'none'
4  version: 0.1.0
5
6  environment:
7    sdk: '>=3.5.0-138.0.dev <4.0.0'
8
9  dependencies:
10    flutter:
11      sdk: flutter
12    shared_preferences: ^2.2.3
13
14  dev_dependencies:
15    flutter_test:
16      sdk: flutter
17    flutter_lints: ^3.0.0
18
19  flutter:
20    uses-material-design: true
21
```

- Para realizar o ato de gravar dados em um arquivo ou ler dados de um arquivo. Para isto são utilizados 2 métodos específicos, **setString** e **getString**.
- **setString**: Este método é usado para armazenar uma String na memória compartilhada (SharedPreferences). Ele requer uma chave e o valor da String a ser armazenada. Se uma String com a mesma chave já existir nas preferências compartilhadas, ela será substituída pelo novo valor.
- **getString**: Este método é usado para recuperar uma String (SharedPreferences) com base na chave fornecida.

Exemplo prático

Notas do Aluno

Nome do aluno
Gargula

Nota 1
8

Nota 2
7

Salvar Dados

Calcular Média

Gargula tem média 7.5. Situação: APROVADO.

Exemplo prático

Arquivo “aluno_app.dart”

Arquivo “main.dart”

```
main.dart > ...
1 import 'package:flutter/material.dart';
2 import 'package:projeto_preferences/screens/aluno_app.dart';
3
4
5 Run | Debug | Profile
6 void main() {
7   runApp(
8     MaterialApp(
9       home: AlunoApp()
10     ) // MaterialApp
11   );
12 }
13
```

```
1 import 'package:flutter/material.dart';
2 import 'package:shared_preferences/shared_preferences.dart';
3
4 class AlunoApp extends StatefulWidget {
5   @override
6   _AlunoAppState createState() => _AlunoAppState();
7 }
8
9 class _AlunoAppState extends State<AlunoApp> {
10   final nomeController = TextEditingController();
11   final nota1Controller = TextEditingController();
12   final nota2Controller = TextEditingController();
13   String resultado = '';
14
15   Future<void> salvarDados() async {
16     final prefs = await SharedPreferences.getInstance();
17
18     String nome = nomeController.text;
19     double nota1 = double.tryParse(nota1Controller.text)!;
20     double nota2 = double.tryParse(nota2Controller.text)!;
21
22     await prefs.setString('nome', nome);
23     await prefs.setDouble('nota1', nota1);
24     await prefs.setDouble('nota2', nota2);
25
26     setState(() {
27       resultado = 'Dados salvos com sucesso!';
28     });
29   }
30
31   Future<void> calcularMedia() async {
32     final prefs = await SharedPreferences.getInstance();
33
34     String nome = prefs.getString('nome')!;
35     double nota1 = prefs.getDouble('nota1')!;
36     double nota2 = prefs.getDouble('nota2')!;
37
38     double media = (nota1 + nota2) / 2;
39
40     String status = media >= 6 ? 'APROVADO' : 'REPROVADO';
41
42     setState(() {
43       resultado = '$nome tem média $media. Situação: $status.';
44     });
45   }
46 }
```

```
46 @override
47 Widget build(BuildContext context) {
48   return Scaffold(
49     appBar: AppBar(title: Text('Notas do Aluno')),
50     body: Padding(
51       padding: const EdgeInsets.all(16.0),
52       child: Column(
53         children: [
54           TextField(
55             controller: nomeController,
56             decoration: InputDecoration(labelText: 'Nome do aluno'),
57           ), // TextField
58           TextField(
59             controller: nota1Controller,
60             decoration: InputDecoration(labelText: 'Nota 1'),
61             keyboardType: TextInputType.number,
62           ), // TextField
63           TextField(
64             controller: nota2Controller,
65             decoration: InputDecoration(labelText: 'Nota 2'),
66             keyboardType: TextInputType.number,
67           ), // TextField
68           SizedBox(height: 20),
69           ElevatedButton(onPressed: salvarDados, child: Text('Salvar Dados')),
70           SizedBox(height: 20),
71           ElevatedButton(
72             onPressed: calcularMedia,
73             child: Text('Calcular Média'),
74           ), // ElevatedButton
75           SizedBox(height: 20),
76           Text(
77             resultado,
78             style: TextStyle(fontSize: 18),
79             textAlign: TextAlign.center,
80           ), // Text
81         ],
82       ), // Column
83     ), // Padding
84   ); // Scaffold
85 }
86
87
88
```



- Para realizar o acesso às APIs, o Flutter utiliza uma lib chamada 'http';
- Para utilizá-la devemos adicionar ela no projeto utilizando o comando:

```
flutter pub add http
```

- Depois a **lib** deve ser importada dentro do projeto, exemplo:

```
import 'package:http/http.dart' as http;
```



- A lib ***http*** possui os seguintes métodos disponíveis para a execução do crud:

✓ ***http.get()***: é usada para enviar uma requisição HTTP GET a um servidor e receber os dados disponibilizados por este servidor. Exemplo:

```
19 var dadosRecebidos = await http.get(Uri.parse('http://127.0.0.1:5000/'));
```

✓ ***http.post()***: é usada para enviar uma requisição HTTP POST a um servidor. Exemplo:

```
await http.post(Uri.parse('http://127.0.0.1:5000/cadastraFuncionario'),  
  headers: {  
    "Accept": "application/json",  
    "content-type": "application/json"  
  },  
  body: json.encode(mapFuncionario),  
  encoding: Encoding.getByName("utf-8"),  
);
```



- A lib ***http*** possui os seguintes métodos disponíveis para a execução do crud:
 - ✓ ***http.put()***: no Flutter é usada para enviar uma requisição HTTP PUT a um servidor. A requisição PUT é utilizada para atualizar ou substituir completamente um recurso existente no servidor com os dados fornecidos no corpo da requisição.

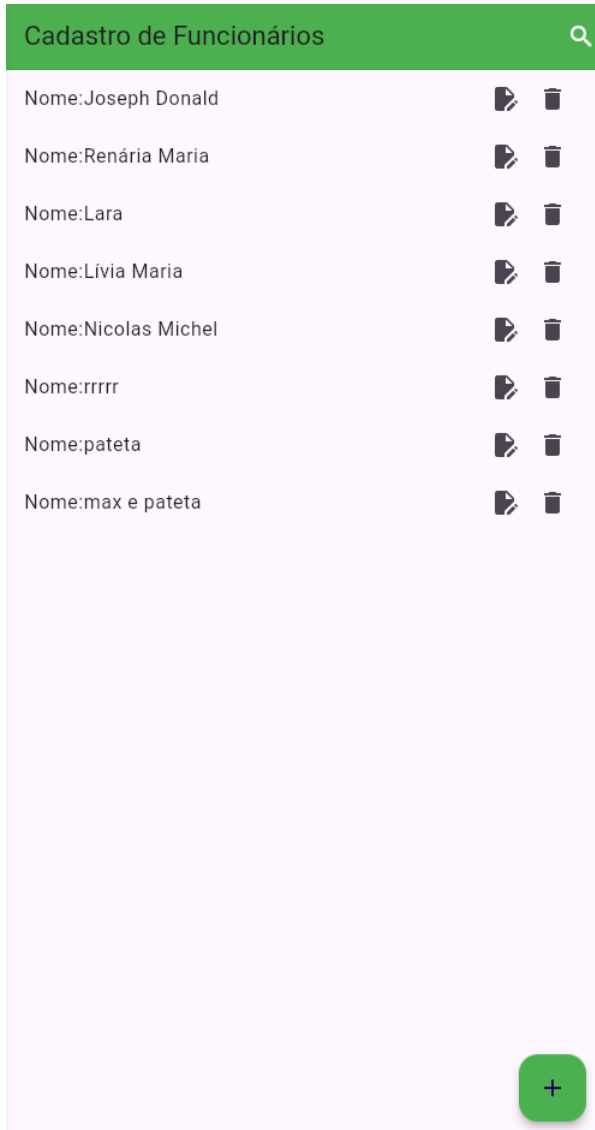
Exemplo:

```
51      await http.put(Uri.parse("http://127.0.0.1:5000/atualizaFuncionario"),
52        headers: {
53          "Accept": "application/json",
54          "content-type": "application/json"
55        },
56        body: json.encode(mapFuncionario),
57        encoding: Encoding.getByName("utf-8"),
58      );
```



- A lib ***http*** possui os seguintes métodos disponíveis para a execução do crud:
 - ✓ ***http.delete()***: no Flutter é usada para enviar uma requisição HTTP DELETE a um servidor. Exemplo:

```
68     await http.delete(Uri.parse('http://127.0.0.1:5000/removerFuncionario'),
69       headers: {
70         "Accept": "application/json",
71         "content-type": "application/json"
72       },
73       body: json.encode(mapFuncionario),
74       encoding: Encoding.getByName("utf-8"),
75     );
```



- Neste exemplo temos um CRUD para um cadastro de funcionários que consome uma API JSON.
- As rotas da API são as seguintes:
 - ☐ Para ler todos os funcionários:
✓ <http://127.0.0.1:5000/>
 - ☐ Pesquisa nomes que contém o termo “jose”
<http://127.0.0.1:5000/pesquisanome/jose>
 - ☐ Cadastra novos funcionários
✓ <http://127.0.0.1:5000/cadastraFuncionario>
 - ☐ Atualiza funcionários
<http://127.0.0.1:5000/atualizaFuncionario>
 - ☐ Remove funcionário
<http://127.0.0.1:5000/removerFuncionario>


```
2 import 'dart:convert';
3 import 'package:http/http.dart' as http;
4 import 'package:flutter/material.dart';
5 import 'package:http/http.dart';
6
7 class Home extends StatefulWidget {
8   const Home({super.key});
9
10  @override
11  State<Home> createState() => _HomeState();
12 }
13
14 class _HomeState extends State<Home> {
15   List listaFuncionarios = [];
16
17   //LENDOS DADOS DE TODOS OS FUNCIONÁRIOS DA API/JSON
18   Future<void> lerFuncionarios() async {
19     var dadosRecebidos = await http.get(Uri.parse('http://127.0.0.1:5000/'));
20     setState(() {
21       listaFuncionarios = jsonDecode(dadosRecebidos.body);
22     });
23   }
24
25   //CADASTRAR FUNCIONÁRIOS PELA API/JSON
26   Future<void> cadastrarFuncionario(String nomeCadastrado) async {
27     var mapFuncionario = {"nomeFuncionario": nomeCadastrado};
28     await http.post(
29       Uri.parse('http://127.0.0.1:5000/cadastraFuncionario'),
30       headers: {
31         "Accept": "application/json",
32         "content-type": "application/json"
33       },
34       body: json.encode(mapFuncionario),
35       encoding: Encoding.getByName("utf-8"),
36     );
37     setState(() {
38       lerFuncionarios();
39     });
40     Navigator.pop(context);
41   }
```

```
42
43 //ATUALIZAR FUNCIONARIO
44 Future<void> atualizarFuncionario(int idRecebido, String nomeRecebido) async {
45   print(nomeRecebido);
46   print(idRecebido);
47   var mapFuncionario = {
48     "idFuncionario": idRecebido,
49     "nomeFuncionario": nomeRecebido
50   };
51   await http.put(Uri.parse("http://127.0.0.1:5000/atualizaFuncionario"),
52     headers: {
53       "Accept": "application/json",
54       "content-type": "application/json"
55     },
56     body: json.encode(mapFuncionario),
57     encoding: Encoding.getByName("utf-8"),
58   );
59   setState(() {
60     lerFuncionarios();
61   });
62   Navigator.pop(context);
63 }
64
65 //REMOVER FUNCIONARIOS PELA API/JSON
66 Future<void> deletarFuncionario(int idRecebido) async {
67   var mapFuncionario = {"idFuncionario": idRecebido};
68   await http.delete(Uri.parse('http://127.0.0.1:5000/removerFuncionario'),
69     headers: {
70       "Accept": "application/json",
71       "content-type": "application/json"
72     },
73     body: json.encode(mapFuncionario),
74     encoding: Encoding.getByName("utf-8"),
75   );
76   setState(() {
77     lerFuncionarios();
78   });
79   Navigator.pop(context);
80 }
```



```
81
82 //PESQUISA POR NOME
83 Future<void> pesquisaNome(String nomePesquisa) async {
84   var dadosRecebidos = await http
85     .get(Uri.parse('http://127.0.0.1:5000/pesquisanome/$nomePesquisa'));
86   setState(() {
87     listaFuncionarios = jsonDecode(dadosRecebidos.body);
88   });
89   Navigator.pop(context);
90 }
91
92 @override
93 void initState() {
94   // TODO: implement initState
95   super.initState();
96   lerFuncionarios();
97 }
98
```

```
99 @override
100 Widget build(BuildContext context) {
101   return Scaffold(
102     floatingActionButton: FloatingActionButton(
103       onPressed: () {
104         TextEditingController controllerNome = TextEditingController();
105         showDialog(
106           context: context,
107           //ALERT PARA ADICIONAR NOVOS FUNCIONÁRIOS
108           builder: (context) => AlertDialog(
109             title: Text("Adicionar Funcionário"),
110             content: TextField(
111               controller: controllerNome,
112               decoration: InputDecoration(
113                 labelText: "Digite o nome do novo funcionário"), // InputDecoration
114             ), // TextField
115             actions: [
116               ElevatedButton(
117                 onPressed: () {
118                   cadastrarFuncionario(controllerNome.text);
119                 },
120                 child: Text("Salvar")), // ElevatedButton
121               ElevatedButton(
122                 onPressed: () {
123                   Navigator.pop(context);
124                 },
125                 child: Text("Cancelar")), // ElevatedButton
126             ],
127           ), // AlertDialog
128         );
129       },
130       child: Icon(Icons.add),
131       backgroundColor: Colors.green,
132     ), // FloatingActionButton

```

```
133 appBar: AppBar(  
134   actions: [  
135     IconButton(  
136       onPressed: () {  
137         TextEditingController controllerNomePesquisa =  
138           TextEditingController();  
139         //ALERT PARA PESQUISAR FUNCIONÁRIOS POR NOME  
140         showDialog(  
141           context: context,  
142           builder: (context) => AlertDialog(  
143             title: Text("Pesquisa por nome"),  
144             content: TextField(  
145               decoration: InputDecoration(  
146                 labelText: "Digite o nome que seja encontrar"), // InputDecoration  
147                 controller: controllerNomePesquisa,  
148               ), // TextField  
149             actions: [  
150               ElevatedButton(  
151                 onPressed: () {  
152                   pesquisaNome(controllerNomePesquisa.text);  
153                 },  
154                 child: Text("Pesquisar")), // ElevatedButton  
155               ElevatedButton(  
156                 onPressed: () {  
157                   Navigator.pop(context);  
158                 },  
159                 child: Text("Cancelar")) // ElevatedButton  
160             ],  
161           ), // AlertDialog  
162         );  
163       },  
164       icon: Icon(  
165         Icons.search,  
166         color: Colors.white,  
167       )) // Icon // IconButton  
168     ],  
169     title: Text("Cadastro de Funcionários"),  
170     backgroundColor: Colors.green,  
171   ), // AppBar
```

```
172 body: ListView.builder(  
173   itemCount: listaFuncionarios.length,  
174   itemBuilder: (context, index) => ListTile(  
175     title: Text("Nome:" + listaFuncionarios[index]["nomeFuncionario"]),  
176     trailing: Row(  
177       mainAxisAlignment: MainAxisAlignment.min,  
178       children: [  
179         IconButton(  
180           onPressed: () {  
181             TextEditingController controllerNomeEdit =  
182               TextEditingController();  
183  
184             controllerNomeEdit.text =  
185               listaFuncionarios[index]["nomeFuncionario"];  
186             //ALERT PARA ATUALIZAR OS DADOS DOS FUNCIONÁRIOS  
187             showDialog(  
188               context: context,  
189               builder: (context) => AlertDialog(  
190                 title: Text("Atualizar funcionário"),  
191                 content: TextField(  
192                   decoration:  
193                     InputDecoration(labelText: "Digite o novo nome"),  
194                   controller: controllerNomeEdit,  
195                 ), // TextField  
196                 actions: [  
197                   ElevatedButton(  
198                     onPressed: () {  
199                       atualizarFuncionario(  
200                         listaFuncionarios[index]["idFuncionario"],  
201                         controllerNomeEdit.text);  
202                     },  
203                     child: Text("Atualizar")), // ElevatedButton  
204                   ElevatedButton(  
205                     onPressed: () {  
206                       Navigator.pop(context);  
207                     },  
208                     child: Text("Cancelar")) // ElevatedButton  
209                 ],  
210               ), // AlertDialog  
211             );  
212           },  
213         ],  
214       ),  
215     ),  
216   ),
```

Código fonte do projeto prático

```
213         icon: Icon(Icons.edit_document)), // IconButton
214     IconButton(
215         onPressed: () {
216             //ALERT PARA REMOVER FUNCIONÁRIOS
217             showDialog(
218                 context: context,
219                 builder: (context) => AlertDialog(
220                     title: Text("Deletando um funcionário"),
221                     content: Text("Deseja remover o funcionário " +
222                         listaFuncionarios[index]["nomeFuncionario"] +
223                         "?"), // Text
224                     actions: [
225                         ElevatedButton(
226                             onPressed: () {
227                                 deletarFuncionario(
228                                     listaFuncionarios[index]["idFuncionario"]);
229                             },
230                             child: Text("Deletar")), // ElevatedButton
231                         ElevatedButton(
232                             onPressed: () {
233                                 Navigator.pop(context);
234                             },
235                             child: Text("Cancelar")), // ElevatedButton
236                     ],
237                 ), // AlertDialog
238             );
239         },
240         icon: Icon(Icons.delete)) // IconButton
241     ],
242 ), // Row
243 ), // ListTile
244 ), // ListView.builder
245 ); // Scaffold
246 }
247 }
248
```